Name:Manjyot Wadhwa
Roll no :65
Div:D15C

Experiment 8

Design and Train a Convolutional Neural Network (CNN) on CIFAR-10 Dataset

---

1. Dataset Source

Dataset Name: **CIFAR-10 Dataset**

Source: Canadian Institute for Advanced Research
Loaded using TensorFlow Keras library:

keras.datasets.cifar10.load_data()

---

2. Dataset Description

The CIFAR-10 dataset consists of colored images classified into 10 categories.

📊 Dataset Size

- 50,000 training images
- 10,000 testing images
- Image size: 32 × 32 pixels
- 3 color channels (RGB)

🎯 Target Classes

1. Airplane
2. Automobile
3. Bird
4. Cat
5. Deer
6. Dog
7. Frog

8. Horse
9. Ship
10. Truck

This is a **multi-class image classification problem**.

3.Mathematical Formulation

3.1 Convolution Operation

A convolution layer applies filters to extract spatial features:

[
Output = (Input * Kernel) + Bias
]

Where:

- Kernel = filter matrix
    - = convolution operation

3.2 Activation Function (ReLU)

$$f(x) = max(0, x)$$

Introduces non-linearity.

3.3 Max Pooling

Reduces spatial dimensions:

$$Output = max(region)$$

Helps reduce computation and overfitting.

3.4 Softmax Output Layer

$$\sigma(z_i) = \frac{e^{z_i}}{\sum e^{z_j}}$$

Used for multi-class classification.

3.5 Loss Function

Sparse Categorical Crossentropy:

$$Loss = -\sum y \log(\hat{y})$$

---

4. Algorithm Limitations

- Requires large computational power
- Training time is higher
- Risk of overfitting
- Sensitive to hyperparameters
- Requires large dataset

---

5. Methodology / Workflow

Step 1: Load Dataset

Loaded CIFAR-10 dataset using TensorFlow.

Step 2: Preprocessing

- Normalized pixel values (0–255 → 0–1)
- No manual feature engineering required

Step 3: Build CNN Architecture

The model consists of:

- Conv2D (32 filters)
- MaxPooling
- Conv2D (64 filters)
- MaxPooling
- Conv2D (64 filters)
- Flatten
- Dense layer (64 neurons)
- Output layer (10 neurons, Softmax)

Step 4: Compile Model

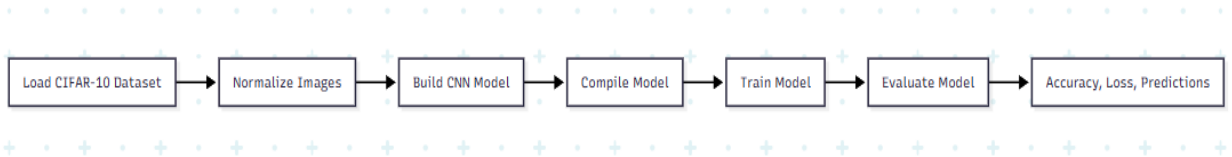
- Optimizer: Adam
- Loss: Sparse Categorical Crossentropy
- Metric: Accuracy

Step 5: Train Model

- Epochs: 5
- Batch Size: 64
- Validation Split: 20%

Step 6: Evaluation

- Test Accuracy
- Accuracy vs Epoch Graph
- Loss vs Epoch Graph
- Random Predictions Visualization
- Classification Report



6.Performance Analysis

📈 Accuracy & Loss Graph

The training accuracy increases over epochs, while validation accuracy stabilizes around 65–70%.

Training loss decreases steadily, indicating learning.

Slight gap between training and validation accuracy indicates mild overfitting.

🎯 Test Accuracy

Test accuracy achieved approximately:

~65–70%

This is expected for a basic CNN trained for 5 epochs on CIFAR-10.

📊 Classification Report

Shows:

- Precision
- Recall
- F1-score
- Support for each class

Some classes (like automobiles, ships) perform better than animals due to clearer features.

Random Predictions

Images were displayed with:

- Green → Correct prediction
- Red → Incorrect prediction

This helps visually analyze model performance.

---

7. Hyperparameter Tuning

Parameters considered:

- Number of convolution layers
- Number of filters (32, 64)
- Kernel size (3×3)
- Batch size (64)

- Number of epochs (5)
- Optimizer (Adam)

Increasing epochs and filters can improve accuracy but increases training time.

---

8. Comparison: ANN vs CNN

ANN	CNN
Flattens image	Preserves spatial structure
Less accurate for images	Better feature extraction
No convolution	Uses filters
More parameters	Efficient parameter sharing

CNN performs significantly better on image data than ANN.

9.CODE

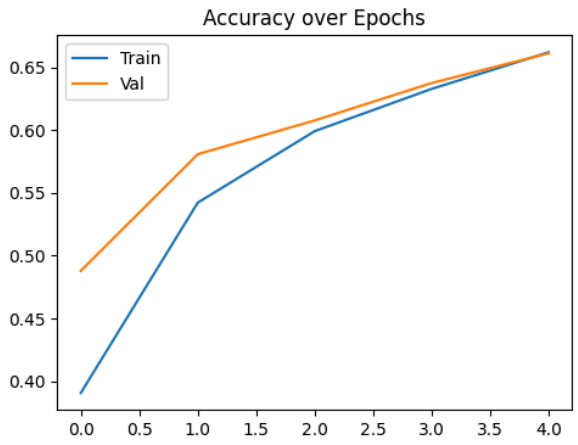
```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from sklearn.metrics import classification_report
import seaborn as sns

# Load Dataset
(x_train, y_train), (x_test, y_test) = keras.datasets.cifar10.load_data()
x_train, x_test = x_train/255.0, x_test/255.0

class_names = ["airplane","automobile","bird","cat",
               "deer","dog","frog","horse","ship","truck"]

# Build CNN
model = keras.Sequential([
    layers.Conv2D(32,(3,3),activation='relu',input_shape=(32,32,3)),
    layers.MaxPooling2D((2,2)),
    layers.Conv2D(64,(3,3),activation='relu'),
    layers.MaxPooling2D((2,2)),
```

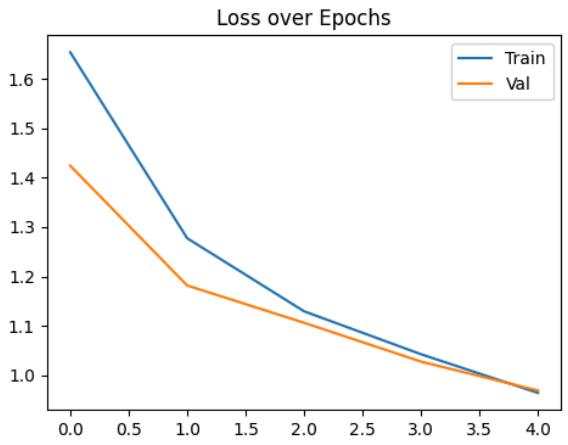
```
    layers.Conv2D(64,(3,3),activation='relu'),
    layers.Flatten(),
    layers.Dense(64,activation='relu'),
    layers.Dense(10,activation='softmax')
])

model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

# Train
history = model.fit(x_train,y_train,
                    epochs=5,
                    batch_size=64,
                    validation_split=0.2,
                    verbose=1)

# Evaluate
test_loss, test_acc = model.evaluate(x_test,y_test)
print("Test Accuracy:", test_acc)

# Predictions
probabilities = model.predict(x_test)
y_pred = np.argmax(probabilities,axis=1)

# --------------------------
# Accuracy & Loss Plot
# --------------------------
plt.figure(figsize=(12,4))

plt.subplot(1,2,1)
plt.plot(history.history['accuracy'],label='Train')
plt.plot(history.history['val_accuracy'],label='Val')
plt.title("Accuracy over Epochs")
plt.legend()

plt.subplot(1,2,2)
plt.plot(history.history['loss'],label='Train')
plt.plot(history.history['val_loss'],label='Val')
plt.title("Loss over Epochs")
plt.legend()

plt.show()

# --------------------------
# Random Predictions
# --------------------------
plt.figure(figsize=(12,6))
```

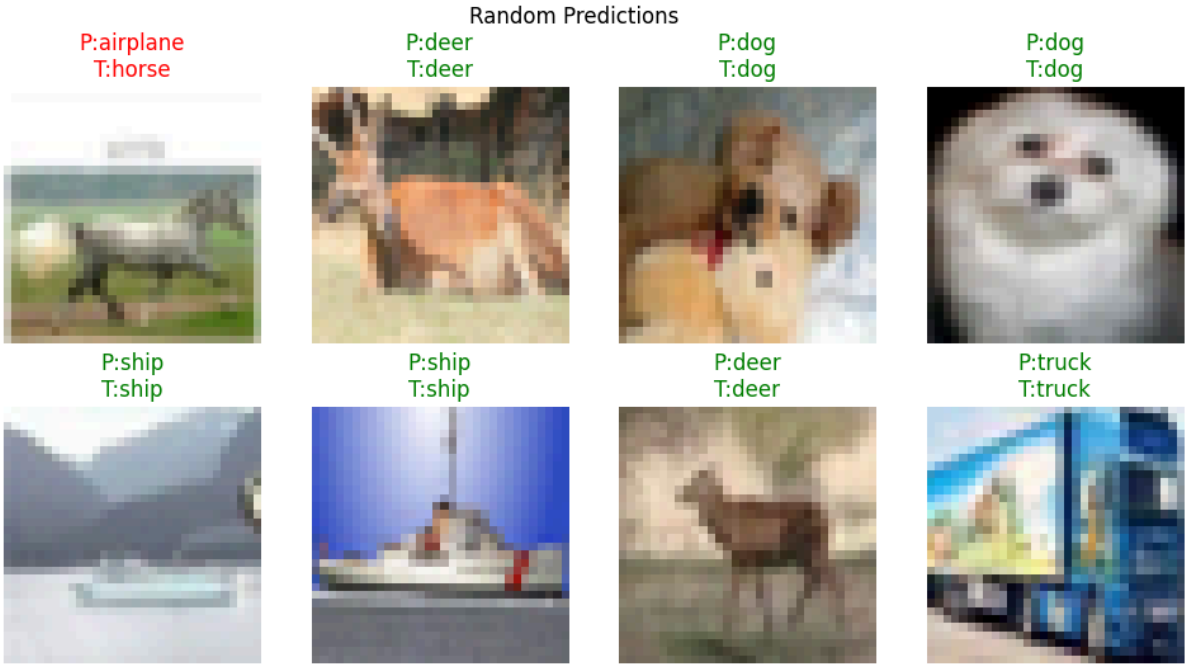
```
for i in range(8):
    idx = np.random.randint(0,len(x_test))
    plt.subplot(2,4,i+1)
    plt.imshow(x_test[idx])
    pred = y_pred[idx]
    true = y_test[idx][0]
    color = "green" if pred==true else "red"
    plt.title(f"P:{class_names[pred]}\nT:{class_names[true]}",color=color)
    plt.axis("off")
plt.suptitle("Random Predictions")
plt.show()

# --------------------------
# Classification Report
# --------------------------
print("\nClassification Report:\n")
print(classification_report(y_test,y_pred))

print("\nExperiment Completed Successfully ✅")
```

10.OUTPUT

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)

Epoch 1/5
625/625 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 60s 93ms/step - accuracy: 0.3033 - loss: 1.8748 - val_accuracy: 0.4879 - val_loss: 1.4240
Epoch 2/5
625/625 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 60s 96ms/step - accuracy: 0.5234 - loss: 1.3224 - val_accuracy: 0.5807 - val_loss: 1.1819
Epoch 3/5
625/625 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 80s 93ms/step - accuracy: 0.5891 - loss: 1.1572 - val_accuracy: 0.6076 - val_loss: 1.1064
Epoch 4/5
625/625 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 59s 94ms/step - accuracy: 0.6278 - loss: 1.0569 - val_accuracy: 0.6374 - val_loss: 1.0279
Epoch 5/5
625/625 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 59s 94ms/step - accuracy: 0.6590 - loss: 0.9722 - val_accuracy: 0.6611 - val_loss: 0.9694
313/313 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 4s 14ms/step - accuracy: 0.6648 - loss: 0.9581
Test Accuracy: 0.6583999991416931
313/313 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 6s 18ms/step
```







Classification Report:

	precision	recall	f1-score	support
0	0.78	0.60	0.67	1000
1	0.74	0.84	0.78	1000
2	0.55	0.54	0.54	1000
3	0.50	0.44	0.46	1000
4	0.59	0.58	0.59	1000
5	0.49	0.67	0.57	1000
6	0.70	0.77	0.73	1000
7	0.80	0.65	0.72	1000
8	0.75	0.79	0.77	1000
9	0.78	0.71	0.74	1000
accuracy			0.66	10000
macro avg	0.67	0.66	0.66	10000

weighted avg	0.67	0.66	0.66	10000

11.Conclusion

A Convolutional Neural Network was successfully designed and trained on the CIFAR-10 dataset.

The CNN effectively extracted spatial features using convolution layers and achieved good classification accuracy.

Although basic architecture was used, the model demonstrated strong performance for multi-class image classification.

Thus, CNN proves to be highly effective for image-based machine learning tasks.